

Report

Automated Bidding Model for Used Car Auctions

Medhavi Sarin

Roll Number: 250123041

May 28, 2026

Contents

1	Introduction	2
2	Data Cleaning	2
2.1	Numeric Columns	2
2.2	Categorical Columns	3
2.3	Summary Table	4
3	Feature Engineering	4
3.1	Feature Correlation Summary	6
4	Exploratory Data Analysis	7
4.1	Depreciation by Body Type	7
4.2	Condition Threshold Analysis	8
4.3	Usage Intensity vs Raw Odometer	9
4.4	Transmission $\times$ Body Type Combinations	10
4.5	Brand $\times$ Condition Heatmap	11
5	Model Selection and Validation	11
5.1	Why LightGBM	11
5.2	Validation Strategy	12
6	Hyperparameter Tuning	12
6.1	Parameter Grid	12
6.2	Results	13
6.3	Default vs Tuned Comparison	13
6.4	Feature Importance	14
7	Bid Sequence	14
7.1	Overview	14
7.2	Layer 1: Dynamic Ceiling	14
7.3	Layer 2: Walk Away Conditions	15
7.4	Layer 3: Geometric Aggression Decay	16
7.5	Layer 4: Snipe Override	16
7.6	Comparison With Flat Increments	17
8	Conclusion	17
8.1	Limitations and Future Work	17

1 Introduction

This report covers the development of an automated bidding agent for Pawn Stars, a wholesale used-car auction firm. The system has two parts: a LightGBM regression model trained on 447,048 historical auction records to predict the final selling price of a used car, and a bidding agent that participates in live auction simulations, reads incoming car data, predicts the car's market value, and places bids using a geometric aggression decay sequence — all within a \$500,000 budget.

Item	Value
Dataset size (original)	447,048 rows, 12 columns
Training set after cleaning	442,585 rows
Features used in model	16 (10 original + 6 engineered)
Final model RMSE	\$2,117
Final model R^2	0.9524

Table 1: Project summary

2 Data Cleaning

2.1 Numeric Columns

Selling Price

The first thing that stood out was selling prices as low as \$1. After checking I got to know that 4,321 rows with prices below \$500, which were most likely data entry errors or system placeholders. Since this is the target variable, filling in fake prices could teach the model completely wrong patterns. Hence I dropped the cars below the \$500 threshold.

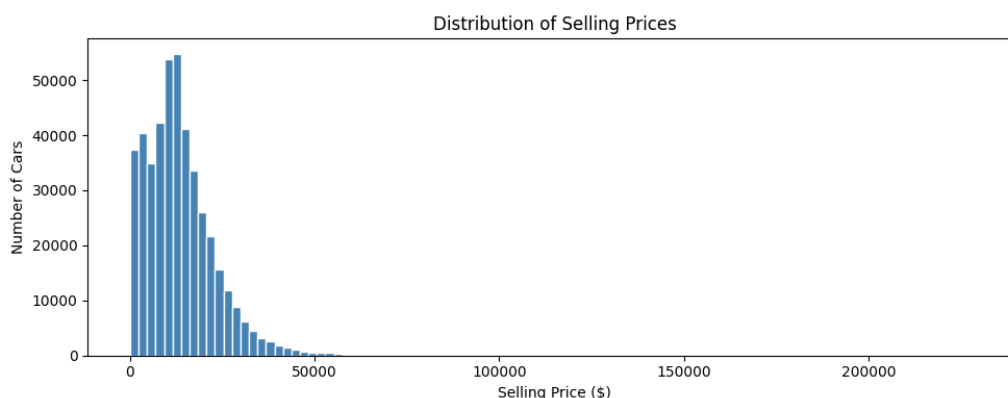


Figure 1: Distribution of selling prices : right-skewed; most cars sell between \$5,000 and \$25,000, with a long tail up to \$230,000.

Odometer

The value 999,999 appeared repeatedly which looked like an entry error so replaced by NaN.

For outlier removal, I first tried using IQR, however, the upper fence would fall around 200,000 miles which would incorrectly remove real cars like a 1998 Civic with 260,000 miles. Hence I decided to use a threshold of 400,000 miles since research suggested the value as a reasonable maximum for any consumer vehicle.

For the remaining missing values, after trying Global median, it felt a better approximation to use group median by make and year. Keeping in mind the agent pipeline rule, the group medians were pre-calculated during training and saved to `imputation_dicts.pkl`, so the agent just does a dictionary lookup.

Condition

9,437 condition values were missing. Using the same group median approach by make and year, luxury brands tend to have higher condition ratings on average, so a global median could underestimate the expected condition of a BMW or Mercedes. IQR analysis suggested no outliers were possible given the fixed 1–5 scale (bounds: 0.45 to 6.45).

Year

The year column had no missing values and a range of 1982 to 2015. This column was dropped from model features later, since `car_age` captures the same information more directly.

2.2 Categorical Columns

Casing

Since "Sedan", "sedan", and " sedan" were being treated as three different categories, I applied lowercase and strip to all 8 categorical columns.

Transmission

52,299 values were missing; around 11.7% of the dataset. Dropping over 50,000 training examples seemed like too large a sacrifice. Seeing that 377,151 cars were automatic vs 13,794 manual in the data, which broadly reflects the US market reality, I filled the missing values with 'automatic'.

Body Type

Pickup trucks appeared under eight different names in the data — "crew cab", "supercab", "extended cab", "regular cab", and several others. These are all the same type of vehicle with different manufacturer-specific terminology. Keeping them as separate categories would create artificial distinctions the model cannot meaningfully learn from, so all were mapped to "pickup". Missing body values were filled with the mode (sedan).

Make, Model, and Trim

Around 8,000–8,500 missing values in each of these columns. Filling with the mode was considered but filling every unknown make with "Ford" would inject false Ford pricing patterns everywhere. Filling with "unknown" felt more honest. The model can then learn whatever pricing patterns exist for unknown-brand cars as a separate group.

Trim had an additional problem of high cardinality: thousands of unique values, many appearing only two or three times. Grouping any trim appearing fewer than 50 times into "other" reduced the unique values down to 474. The threshold of 50 was chosen somewhat arbitrarily — it seemed like the minimum number of appearances needed for the model to pick up a real pattern.

Color, Interior, and State

Color and interior had 20,361 and 14,155 missing values respectively. Both were filled with "unknown" — color does not seem to strongly drive price, and guessing felt unjustified. State had no missing values and 38 unique values, so nothing needed to be done.

2.3 Summary Table

Column	Problem	Method	Justification
sellingprice	Values below \$500	Drop rows	Cannot impute target variable
odometer	999999 placeholder + outliers + missing	Replace → cap 400k → group median	IQR too aggressive on skewed data; group median respects make/year
condition	9,437 missing	Group median (make + year)	Luxury brands rate differently than economy brands
year	None	No action	Already clean; replaced by car_age
All text columns	Inconsistent casing	Lowercase + strip	Prevents duplicate categories
transmission	52,299 missing	Fill "automatic"	Overwhelmingly automatic in US wholesale market
body	8 names for pickups + missing	Map variants → pickup + fill mode	Same vehicle, different manufacturer naming
make / model / trim	Missing + high cardinality	Fill "unknown" + group rare → "other"	Cannot guess brand; rare trims unlearnable
color / interior	Missing	Fill "unknown"	Weak price signal, guessing not justified
state	None	No action	Already clean

Table 2: Data cleaning decisions

3 Feature Engineering

After cleaning, the existing columns still felt like they were leaving useful information on the table. Raw year, for instance, is just a number — the model has no way to directly interpret what "2012" means in terms of depreciation. Odometer tells total miles but

nothing about how those miles were driven. This section covers six features created to address these gaps. Not all of them turned out equally useful — transmission alone, for example, ended up near the bottom of feature importance. All six work on a single row with no dataset-level operations, for compatibility with the live auction agent.

Feature 1: car_age

$$\text{car_age} = \text{AUCTION_YEAR} - \text{year}, \quad \text{AUCTION_YEAR} = 2016$$

The anchor year was set to 2016 because the dataset contains records up to 2015, suggesting these auctions happened around that time.

Correlation with selling price: -0.568

Feature 2: usage_intensity

$$\text{usage_intensity} = \frac{\text{odometer}}{\text{car_age} + 1}$$

Two cars with the same odometer reading can be in very different states depending on how old they are. Odometer alone treats these identically. The $+1$ in the denominator avoids division by zero for brand new cars.

The correlation (-0.345) appears weaker than odometer alone (-0.588), the point of usage intensity is not to replace odometer but to add information that odometer misses .

Correlation with price: -0.345

Feature 3: age_odometer_interaction

$$\text{age_odometer_interaction} = \text{car_age} \times \text{odometer}$$

It tries to capture the combined effect of age and mileage.

Correlation with price: -0.558

Feature 4: is_luxury

$$\text{is_luxury} = \begin{cases} 1 & \text{if make} \in \{\text{bmw, mercedes-benz, audi, lexus, cadillac,} \\ & \text{lincoln, infiniti, acura, land rover,} \\ & \text{porsche, jaguar, volvo, genesis}\} \\ 0 & \text{otherwise} \end{cases}$$

The make column already captures brand information, so in theory this feature is somewhat redundant. The reason for including it was to give the model a direct signal about the luxury segment rather than relying on it to learn this pattern slowly from individual make values. Although it ended up near the bottom of the feature importance chart, likely because make and model already carry most of this information.

Correlation with price: $+0.305$

Feature 5: trans_body

$$\text{trans_body} = \text{transmission} + \text{"_"} + \text{body}$$

Example: "automatic_suv", "manual_sedan". EDA suggested that transmission and body type interact. Keeping them as separate columns forces the model to discover this relationship indirectly.

Feature 6: condition_age_ratio

$$\text{condition_age_ratio} = \frac{\text{condition}}{\text{car_age} + 1}$$

The idea is that condition means something different depending on the age of the car. The same score on a 15-year-old car suggests the owner maintained it carefully over a long period, which buyers tend to value. Dividing by age + 1 rewards well-maintained older cars with a higher ratio. This turned out to be the strongest feature in the dataset, which was not entirely expected going in.

Correlation with price: +0.595

3.1 Feature Correlation Summary

Feature	Correlation with Price	Type
condition_age_ratio	+0.595	Engineered
odometer	−0.588	Original
car_age	−0.579	Engineered
age_odometer_interaction	−0.558	Engineered
usage_intensity	−0.345	Engineered
is_luxury	+0.305	Engineered

Table 3: condition_age_ratio (+0.595) came out stronger than every original column, including raw odometer (−0.588), which was not entirely expected.

4 Exploratory Data Analysis

4.1 Depreciation by Body Type

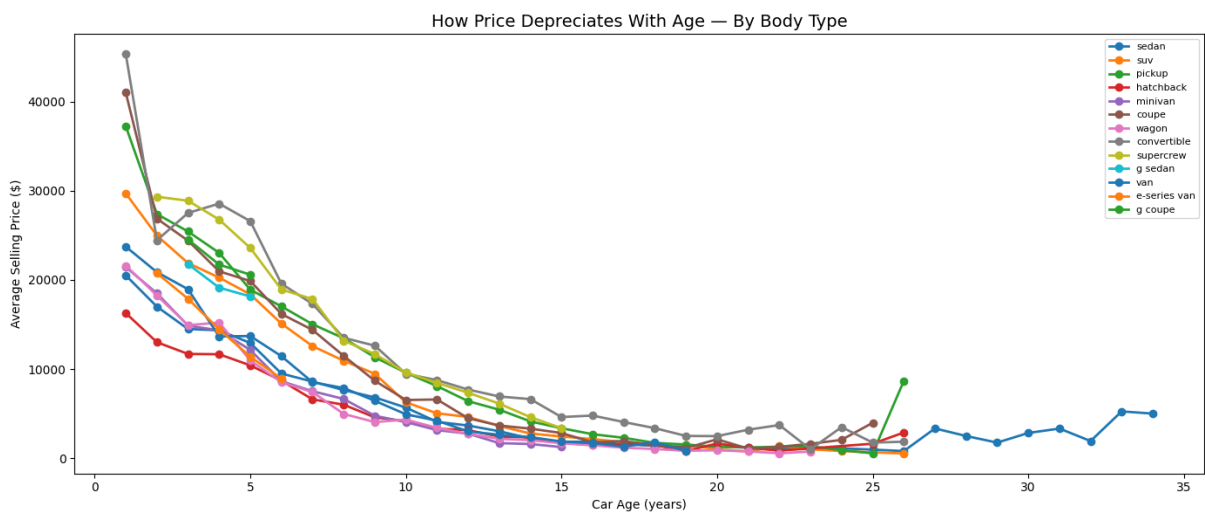


Figure 2: Average selling price vs car age, grouped by body type.

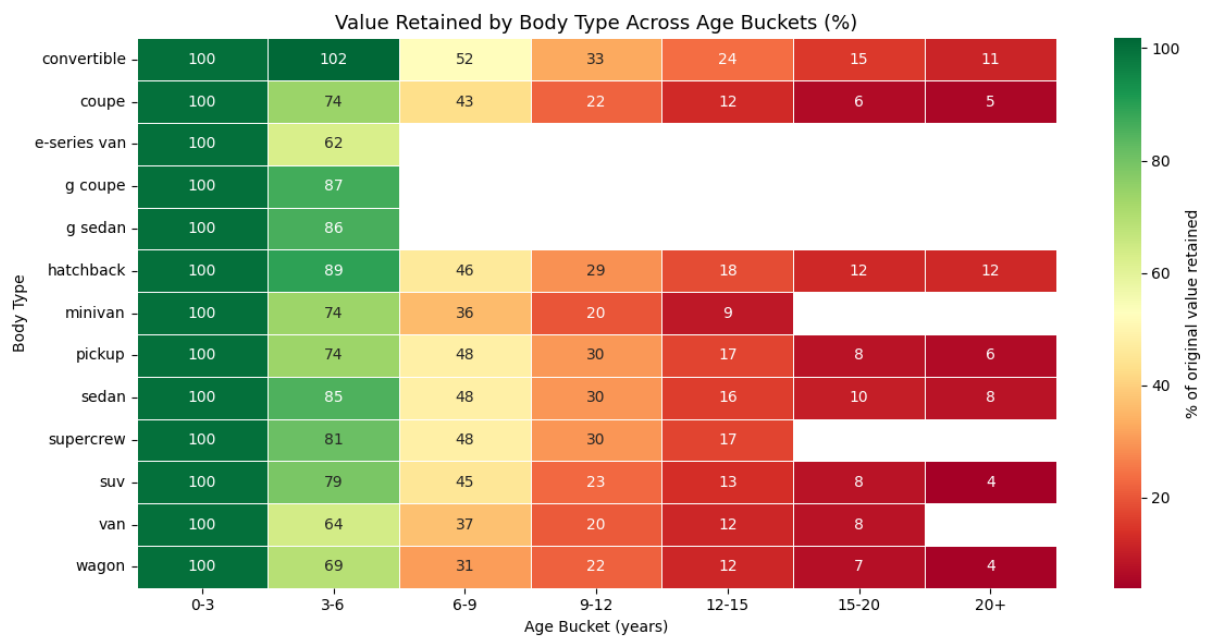


Figure 3: Percentage of original value retained across age buckets.

Depreciation doesn't look linear. It likely follows an exponential decay concentrated in the first 8 years. After year 8 the curves flatten, and by year 20 all body types converge to a roughly \$1,000–\$3,000 floor regardless of type.

Convertibles show 102% value retention at 3–6 years, they briefly appreciate before collapsing which might be due to a supply effect, since relatively few used convertibles in that age range appear on the market at any given time. More surprising was the hatchback

finding: hatchbacks retain 89% of value at 3–6 years compared to SUVs at only 79%, which is against the common assumption that SUVs hold value best. The likely explanation is that manufacturers produce SUVs in very high volumes, flooding the used market and suppressing prices, whereas hatchback buyers tend to pay based on practicality rather than status, not freshness premium.

The bidding implication is that the agent should bid more confidently on trucks and hatchbacks under 8 years old, and treat convertibles with caution unless they are nearly new.

4.2 Condition Threshold Analysis

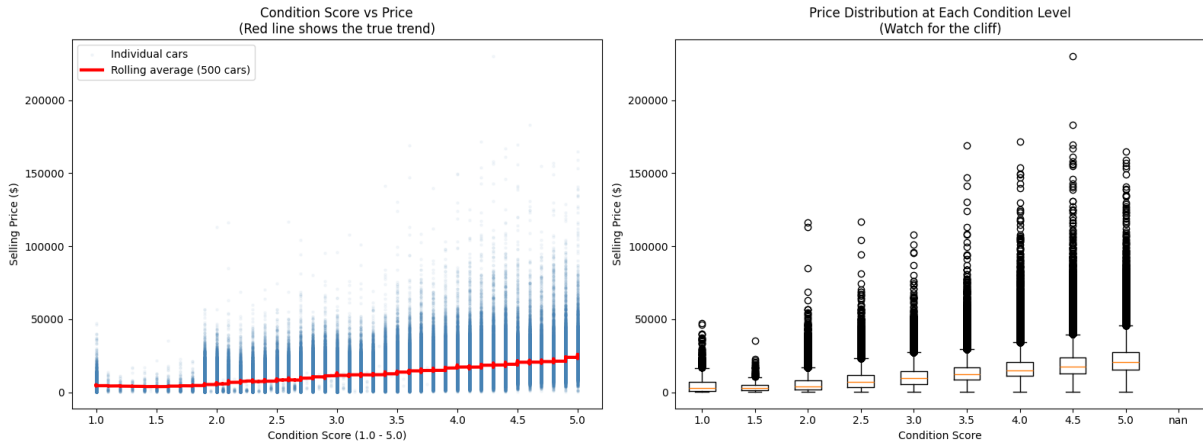


Figure 4: Left: Scatter plot of condition score vs price with a rolling average line (500-car window). Right: Box plot showing price distribution at each condition level.

Price does not respond linearly to condition. The rolling average line is nearly flat from condition 1.0 to 2.5, with prices ranging between roughly \$4,242 and \$8,844. Meaningful acceleration only begins above 2.5.

An interesting edge case: condition 1.5 (\$4,242) prices lower than condition 1.0 (\$5,160). Cars in the 1.5 range seem to be too broken to drive but too functional for pure parts buyers. Above condition 3.5, the box plot shows exploding price variance, which suggests condition interacts strongly with make and body type at higher scores.

Below condition 2.5, pricing is suppressed and unpredictable enough that bidding confidently is difficult. Maximum confidence should only be applied above 3.5.

4.3 Usage Intensity vs Raw Odometer

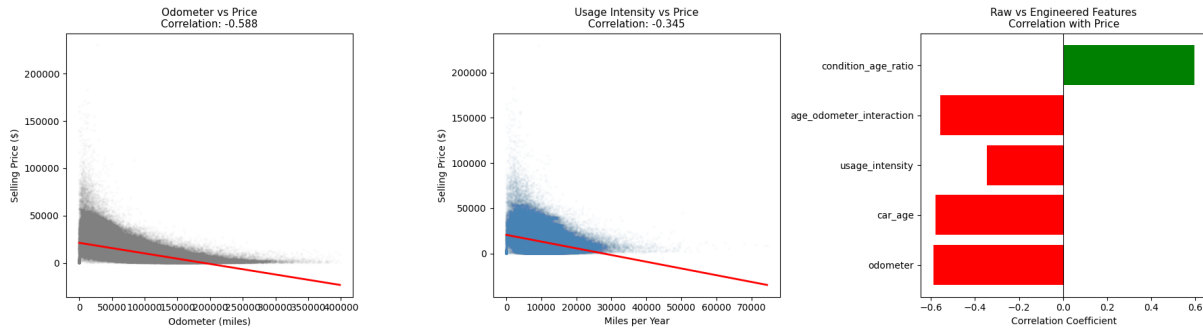


Figure 5: Correlation comparison: raw odometer vs engineered features.

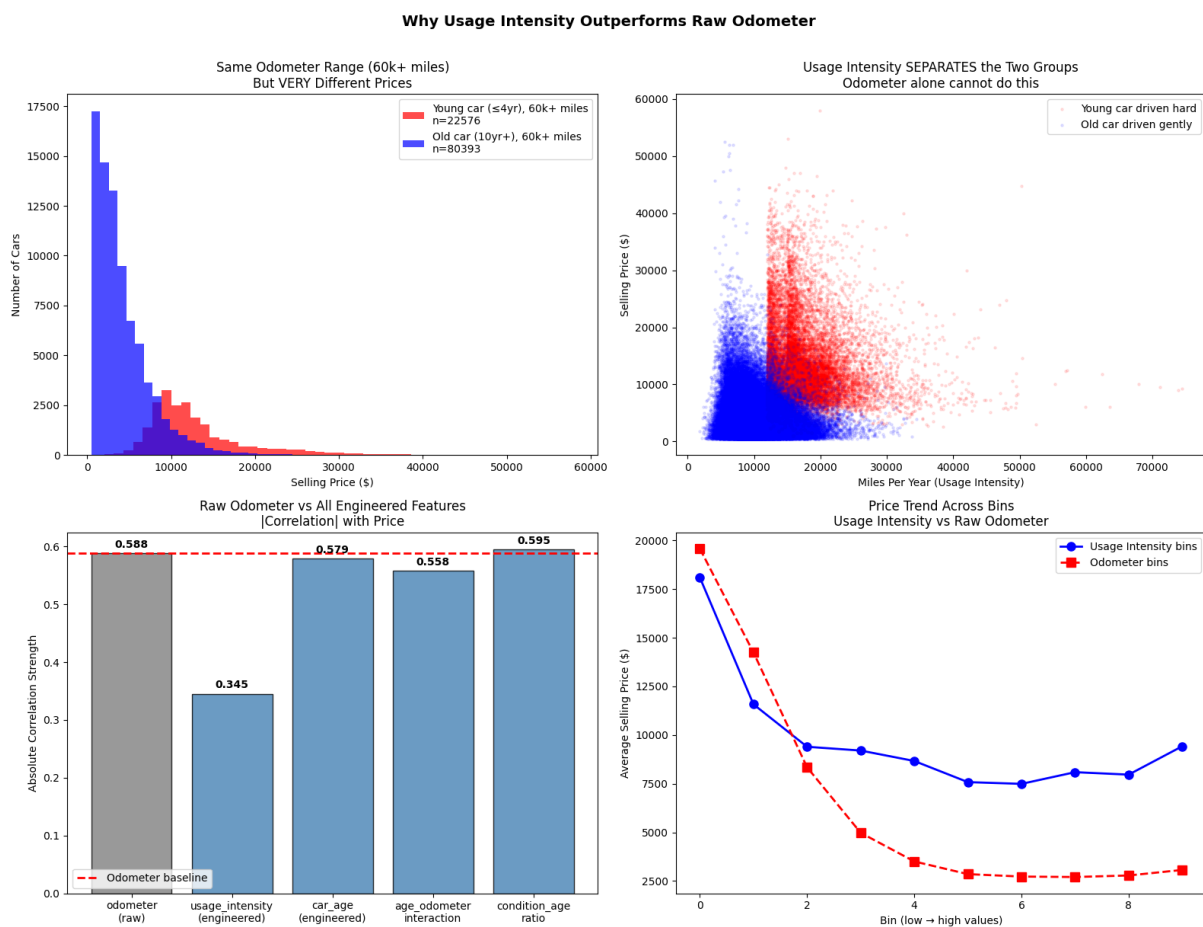


Figure 6: Top left: Two groups of cars with identical 60k+ mile odometers show different prices based on age. Top right: Usage intensity separates these groups. Bottom left: Engineered features and raw odometer. Bottom right: Usage intensity bins plateau while odometer bins collapse.

4.4 Transmission × Body Type Combinations

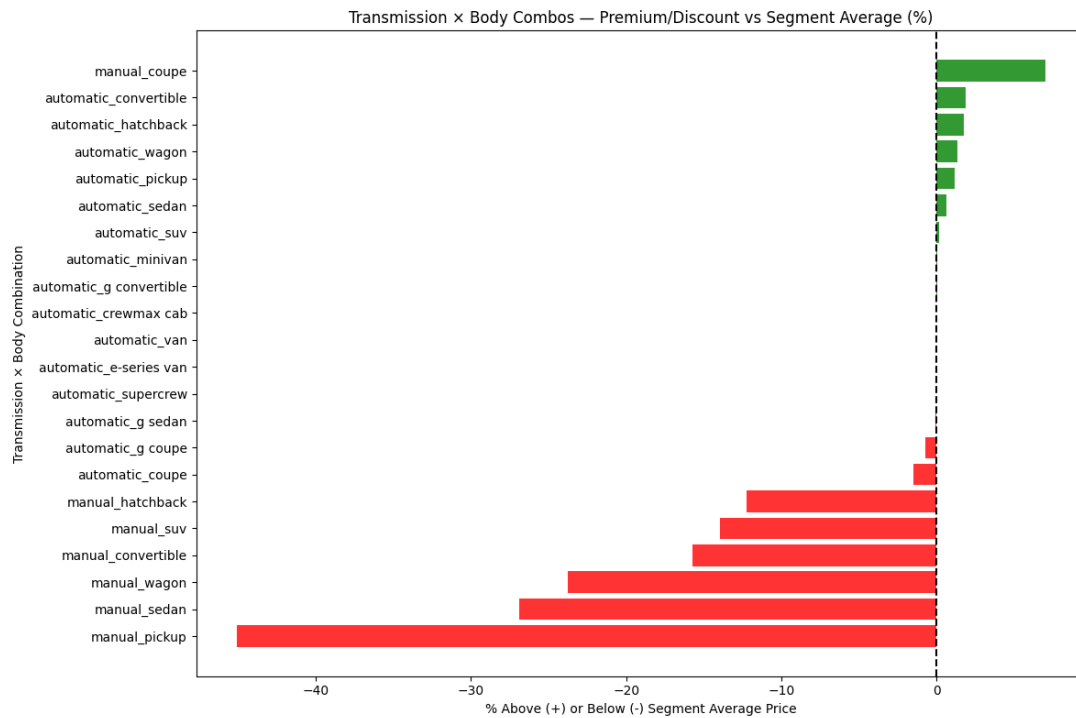


Figure 7: Premium or discount versus segment average for each transmission-body combination. Green bars beat the segment average; red bars underperform.

Automatic transmission commands a premium across almost all body types. Manual coupe is the only combination that beats the segment average (+7%). The reason could be that enthusiast buyers specifically seek manual transmissions in sporty cars. Manual pickups in daily traffic can be a tight spot to be in, hence appearing at the bottom of dataset.

4.5 Brand × Condition Heatmap

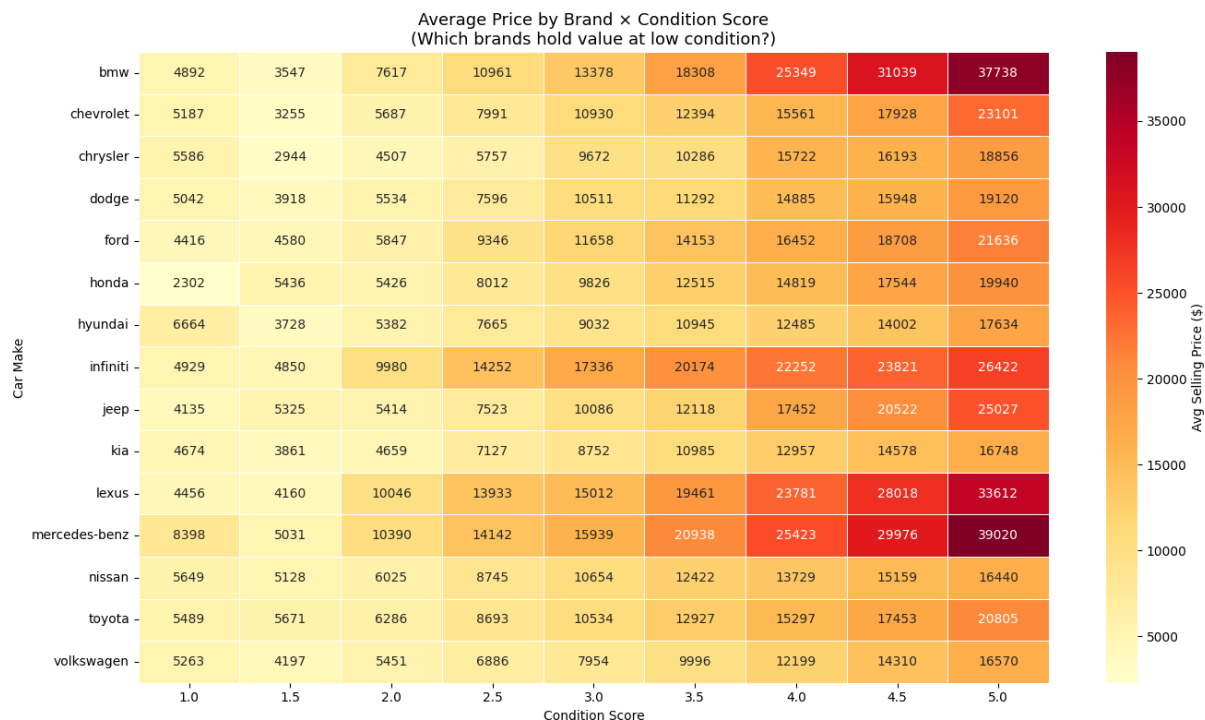


Figure 8: Average selling price for each brand at each condition score level.

The heatmap reveals that the luxury premium is condition-dependent rather than constant. At condition 1.0, Mercedes (\$8,398) is only about 3.6 times more expensive than Honda (\$2,302). At condition 5.0, the gap widens. The premium appears to activate most strongly above condition 3.5, and below 2.5 luxury brands offer relatively little pricing advantage over economy brands.

Learning from the analysis, we should bid aggressively on luxury cars when condition is above 3.5, and treat them with more caution below 2.5 where the premium largely disappears.

5 Model Selection and Validation

5.1 Why LightGBM

Gradient boosting builds trees one at a time, where each new tree focuses on correcting the errors made by the previous ones.

Several models were considered. Linear and polynomial regression were ruled out early since EDA showed that price follows non-linear patterns with condition thresholds and complex brand interactions that a straight line simply cannot capture. Random Forest with 442,000 rows, training would have taken hours. XGBoost was closer to viable but grows trees level by level, which is less efficient than LightGBM's leaf-wise approach. LightGBM also groups data into histogram buckets before searching for splits, which reduces computation significantly. The training completed in around 20 minutes.

5.2 Validation Strategy

80/20 train-test split and `random_state=42`. 354,068 training rows and 88,517 validation rows. The fixed random state ensures that running the same code again produces identical results, which makes the default vs tuned comparison fully fair.

Two metrics used. RMSE (Root Mean Squared Error) measures average dollar error per car and is the most meaningful metric for a bidding agent — it directly tells the error margin in dollars. R^2 measures what fraction of price variation the model explains, which gives a sense of overall fit quality.

6 Hyperparameter Tuning

RandomizedSearchCV was used after failing to use a full GridSearchCV. The complete parameter grid has 1,296 possible combinations, and testing all of them with 3-fold cross validation would require 3,888 model fits. RandomizedSearchCV tests 50 random combinations (150 fits total), which covers the important regions of the search space reducing computation. Cross validation with `cv=3` splits the training data into three parts, trains on two and validates on one, rotates, and averages the score.

Setting	Value	Why
<code>n_iter</code>	50	50 random combinations from the full grid
<code>cv</code>	3	3-fold cross validation
<code>scoring</code>	<code>neg_root_mean_squared_error</code>	Minimize RMSE
<code>n_jobs</code>	4	4 CPU cores in parallel
<code>random_state</code>	42	Reproducible results

Table 4: RandomizedSearchCV settings

6.1 Parameter Grid

Parameter	Values tried	What it controls
<code>n_estimators</code>	300, 500, 1000	Number of trees
<code>learning_rate</code>	0.01, 0.05, 0.1	How much each tree contributes
<code>max_depth</code>	6, 8, 12, -1	Maximum depth of each tree. -1 means unlimited.
<code>num_leaves</code>	31, 63, 127, 255	Maximum leaves per tree
<code>subsample</code>	0.7, 0.8, 1.0	Fraction of rows each tree sees. Lower values add randomness and can prevent overfitting.
<code>colsample_bytree</code>	0.7, 0.8, 1.0	Fraction of features each tree sees. Forces the model to sometimes rely on less dominant features.

Table 5: Hyperparameter grid

One thing I missed earlier is the relationship between `max_depth` and `num_leaves`. The maximum possible leaves in a tree is $2^{\text{max_depth}}$. If `max_depth` = 4 and `num_leaves` = 127, the tree physically cannot grow beyond 16 leaves — the 127 setting is wasted.

6.2 Results

Parameter	Best value found
<code>n_estimators</code>	1000
<code>learning_rate</code>	0.1
<code>max_depth</code>	12
<code>num_leaves</code>	127
<code>subsample</code>	1.0
<code>colsample_bytree</code>	1.0

Table 6: Best hyperparameters found

Both `subsample` and `colsample_bytree` settled at 1.0, meaning no row or feature subsampling helped.

6.3 Default vs Tuned Comparison

Metric	Default Model	Tuned Model	Improvement
RMSE	\$3,173	\$2,117	−33.3%
R^2	0.8931	0.9524	+0.0593

6.4 Feature Importance

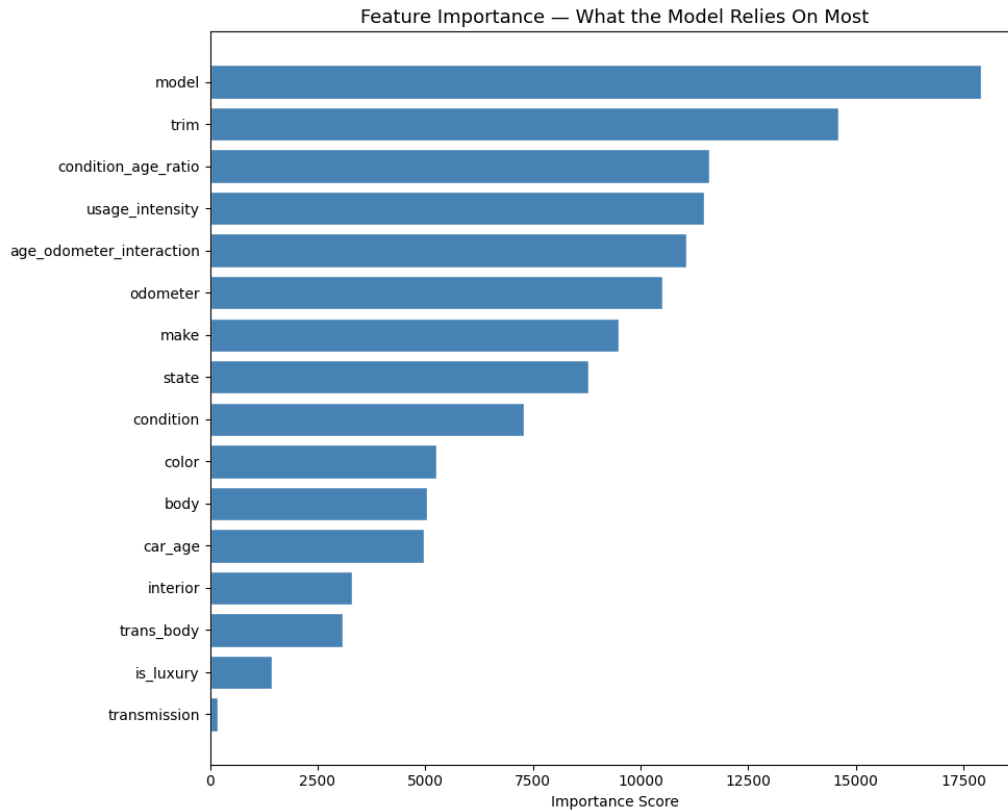


Figure 9: model and trim ranking highest confirms that specific vehicle identity dominates auction pricing

7 Bid Sequence

7.1 Overview

The next bid is always a pure function of four inputs: `self.predicted_value` (what the model thinks this car is worth), `self.bankroll` (how much money remains), `current_highest_bid` (the current auction price), and `self.round_number` (n , which round of bidding we are in). Same inputs always produce the same bid.

7.2 Layer 1: Dynamic Ceiling

The ceiling is the maximum the agent will ever pay for a car. It is computed at the start of each car's bidding and does not change while bidding on that car.

$$\text{max_bid} = \text{predicted_value} \times (1 - \text{margin})$$

I tried keeping the margin value dynamic by including factors like the car's predicted value tier. More expensive cars get a slightly smaller margin because more absolute profit room.

Predicted Value	Base Margin	Ceiling
$\geq \$20,000$	8%	92% of value
$\$10,000 - \$19,999$	10%	90% of value
$< \$10,000$	12%	88% of value

Table 7: Base margin by car value tier.

The base margin is also adjusted based on the current financial state. When the bankroll is running low, a penalty is added to require a larger profit cushion before bidding. If the agent has been doing well and has accumulated estimated profit, a small bonus reduces the margin slightly, allowing marginally more aggressive bids.

Bankroll Remaining	Penalty Added	Why
$< 20\%$	+7%	Less than \$100,000 left — need larger profit guarantee
20%–40%	+3%	Getting low — moderate caution
$> 40\%$	0%	Comfortable — no adjustment

Table 8: Bankroll-based margin penalty.

$$\text{profit_bonus} = \min\left(\frac{\text{estimated_profit}}{1,000,000}, 0.04\right)$$

Every \$10,000 of accumulated estimated profit reduces the margin by 1%, capped at 4% to prevent recklessness.

$$\begin{aligned}\text{margin} &= \text{base_margin} + \text{bankroll_penalty} - \text{profit_bonus} \\ \text{margin} &= \text{clamp}(\text{margin}, 0.06, 0.22)\end{aligned}$$

The clamp between 6% and 22% prevents extreme behaviour in either direction.

7.3 Layer 2: Walk Away Conditions

Four conditions cause the agent to return 0.0 immediately. First, if the current highest bid is already at or above the ceiling, there is no room to profit so the agent walks away. Second, a per-car budget cap prevents spending too large a fraction of the remaining bankroll on a single car.

Bankroll Remaining	Cap	Why
$> 60\%$	20% of bankroll	Rich — can afford larger individual purchases
30%–60%	12% of bankroll	Medium — balanced
$< 30\%$	7% of bankroll	Poor — protect remaining funds

Table 9: Per-car budget cap. These values ensure the agent can always afford at least 5, 8, or 14 more cars respectively.

Third, if the current bid exceeds the entire remaining bankroll. Fourth, a session spending cap applies once total spending exceeds 65% of the initial budget (\$325,000 spent): at that point, the agent only bids on cars predicted above \$25,000, preserving the remaining \$175,000 for high-value opportunities later in the auction.

7.4 Layer 3: Geometric Aggression Decay

$$\begin{aligned}\text{gap}(n) &= \text{max_bid} - \text{current_highest_bid} \\ \text{increment}(n) &= \max\left(\text{gap}(n) \times \frac{X}{n}, \text{min_increment}\right) \\ \text{next_bid}(n) &= \min(\text{current_bid} + \text{increment}(n), \text{max_bid})\end{aligned}$$

Where n is the round number and X is an aggression constant that scales with the remaining bankroll.

Bankroll Ratio	X	Min Increment	Why
> 60%	0.45	\$400	Rich — large early jumps signal strength
30%–60%	0.30	\$200	Medium — balanced aggression
< 30%	0.15	\$100	Poor — minimal increments

Table 10: Aggression constant by bankroll level.

The $1/n$ in the formula causes the decay — as rounds increase, the fraction X/n shrinks, producing smaller increments each round. Early rounds are aggressive (the round 1 jump is nearly half the gap), which signals high valuation to rivals and often causes price-sensitive agents to exit. Late rounds are gentle, protecting the profit margin as the ceiling approaches. The sequence converges: as $n \rightarrow \infty$, $\text{increment} \rightarrow 0$, so bids approach max_bid asymptotically without crossing it. The minimum increment floor is kept to prevent trivially small bids in late rounds.

7.5 Layer 4: Snipe Override

If $\text{round_number} \geq 3$ and $\text{gap} > \text{max_bid} \times 0.20$, the increment becomes $\text{gap} \times 0.75$ instead of the decay formula. The idea is that if we are already in round 3 and the gap is still more than 20% of the ceiling, rather than slowly incrementing toward the ceiling, we jump most of the remaining gap and close the deal. The jump is set at 75% rather than 100% to leave a small buffer for the $\text{min}()$ safety check.

7.6 Comparison With Flat Increments

Property	Geometric Decay	Flat \$500 Increment
Round 1 signal	Large jump deters weak rivals	Small jump — easily ignored
Late round safety	Tiny increments protect margin	Same \$500 every round — can overshoot
Convergence	Approaches ceiling asymptotically	May stop far below ceiling
Predictability	Variable jumps — ceiling is hard to guess	Rivals can calculate ceiling after 2 rounds
Deal closing	Snipe closes deals in 1 move	May need 20+ rounds

Table 11: Geometric decay vs flat increment.

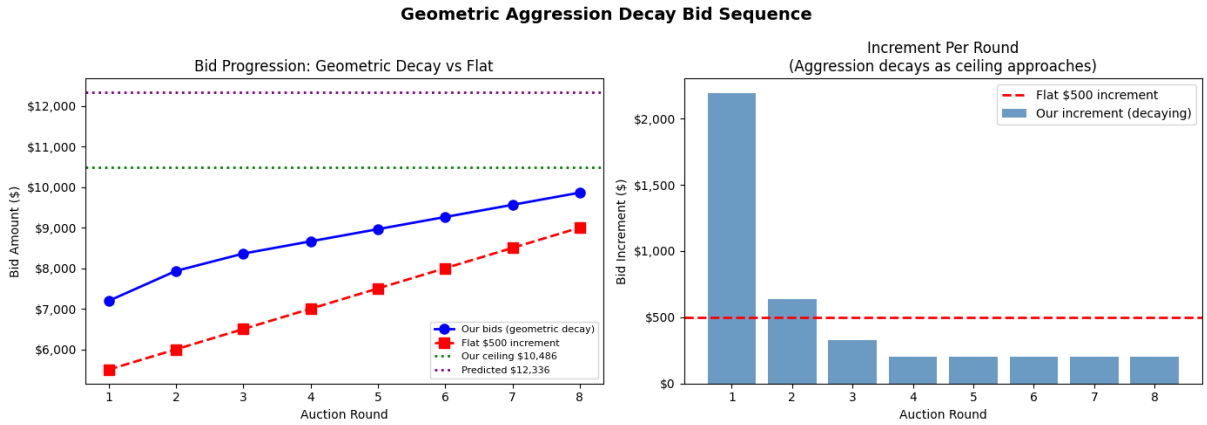


Figure 10: Left: Bid progression over 8 rounds — geometric decay (blue) reaches the ceiling faster than flat increment (red). Right: Increment size per round — the round 1 jump of roughly \$2,100 is about 4 times the flat \$500 baseline, decaying to the minimum floor by round 3.

8 Conclusion

8.1 Limitations and Future Work

Limitations

- **Static Macroeconomic Isolation:** We have not considered that in reality, wholesale used-car prices are highly sensitive to external macroeconomic variables that are not present in the dataset, such as prevailing interest rates and fuel prices.
- **Information Loss in High-Cardinality Data:** By grouping rare `trim` variants (appearing fewer than 50 times) into an "other" category to prevent overfitting, the

model loses the ability to accurately price rare enthusiast trims that might command a premium.

- **Dimensionality of Condition:** The 'condition' column ranges between 1.0 to 5.0. Unstructured data like inspector text notes and lot photographs could help in knowing the type of damage.

Future Improvements

- **Quantile Regression for Confidence Intervals:** Rather than predicting a single mean price, the LightGBM objective function could be shifted to quantile regression (e.g., predicting the 10th and 90th percentiles). This way, the bidding agent can scale its margin requirements based on the statistical confidence like bidding aggressively on tight intervals and conservatively on wide intervals.
- **Bayesian Hyperparameter Optimization:** While `RandomizedSearchCV` was efficient, upgrading to a Bayesian optimization framework (such as `Optuna`) would allow the hyperparameter search to learn from previous iterations, likely finding a lower RMSE with less computational overhead.
- **External Feature Integration:** The dataset could be enriched by merging external time-series data, such as historical regional gas prices, to allow the model to capture macroeconomic shifts (e.g., the shifting premium between SUVs and hatchbacks during fuel spikes).